



Laboratorio 3

Automatización de procesos con n8n

ICN-292 — Sistemas de Información para la Gestión

Alumna: Antonia Mena

Paralelo: 101

Fecha: 11-09-2026

Resumen ejecutivo

El presente informe detalla el diseño y la automatización de un sistema de *Triage* para las solicitudes de devolución del canal e-commerce de AndesHogar SpA utilizando la plataforma n8n, utilizando un **S de 220, un U de 50000 y un D de 7 días**. El objetivo del proyecto fue resolver el cuello de botella actual en el equipo de servicio al cliente, donde el procesamiento manual de los casos genera un tiempo de primera respuesta de tres días hábiles, incumpliendo la política corporativa de 24 horas.

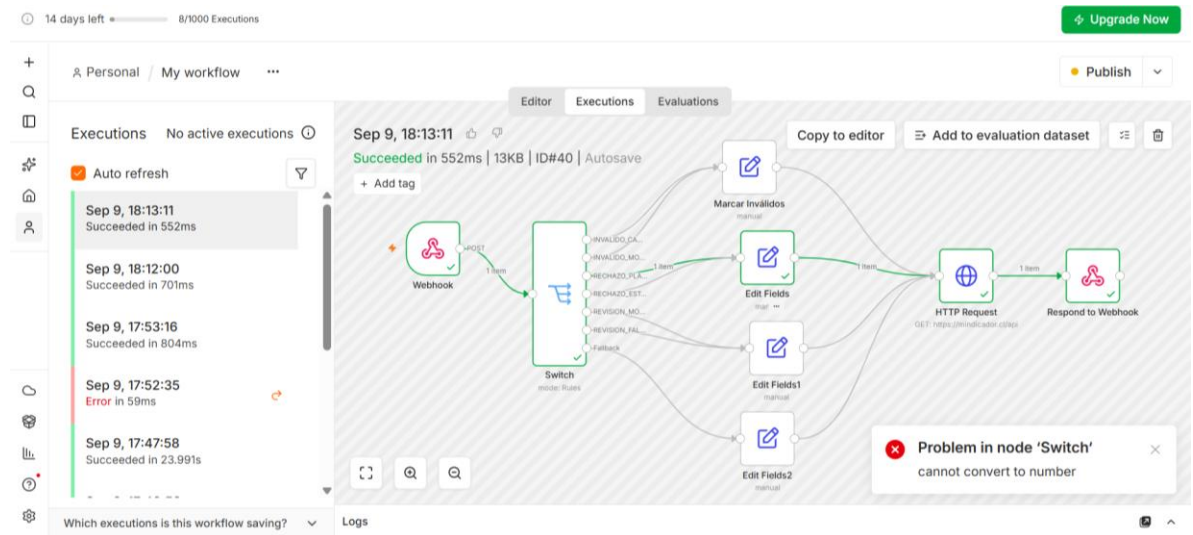
Para solucionar esto, se construyó un producto mínimo viable (MVP) que automatiza la decisión inicial (aprobar, enviar a revisión física o rechazar) en milisegundos. El proyecto abarcó la configuración del flujo de enrutamiento principal (Parte A), la automatización de reportes acumulados diarios (Parte B) y la ejecución de pruebas de estrés y casos borde (Parte C). Adicionalmente, se desarrolló una prueba de concepto mediante programación en python (Nodo Code) para evaluar la escalabilidad técnica de la lógica de negocio.

Recomendación sobre el producto mínimo viable (MVP) Si bien el MVP logra reducir el tiempo de *triage* de tres días a un procesamiento instantáneo y demuestra viabilidad técnica, no se recomienda su despliegue como sistema definitivo en producción sin antes realizar integraciones estructurales. Para que esta herramienta alcance un estándar empresarial robusto en AndesHogar, se requiere implementar las siguientes mejoras:

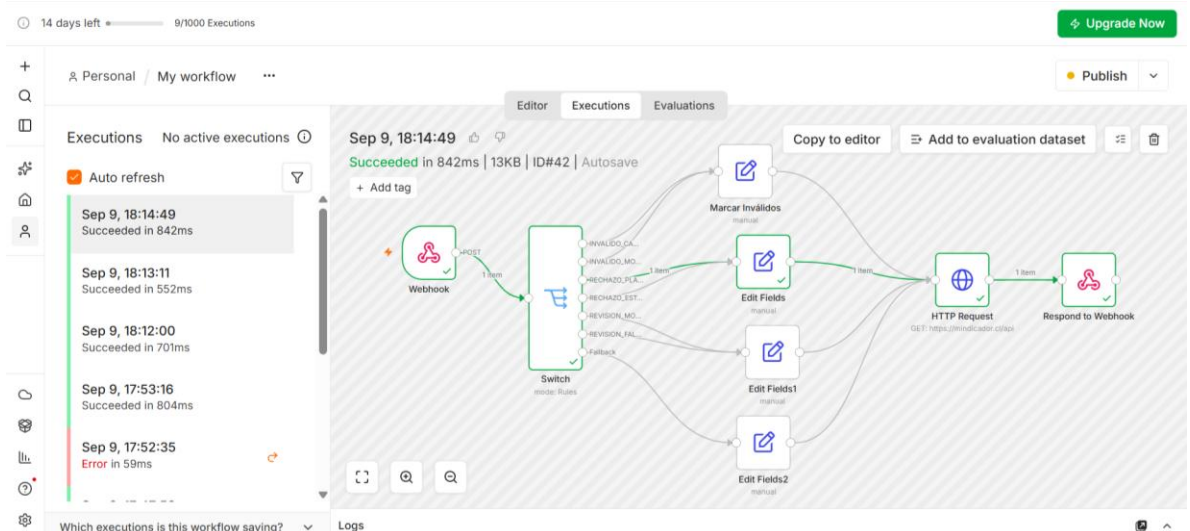
1. **Validación Semántica (Mitigación de Falsos Positivos):** El MVP actual procesa solicitudes basándose únicamente en la sintaxis de los datos. Se debe integrar un nodo de consulta que se conecte directamente al ERP o base de datos maestra de la empresa para validar la existencia real del producto (SKU) y la compra asociada, evitando inconsistencias operativas.
2. **Parametrización Dinámica:** Los límites críticos del negocio (umbral de \$50.000 y 7 días de plazo) están insertos como texto estático en la lógica del flujo (*hardcoding*). En un entorno definitivo, estos parámetros deben consumirse desde un mantenedor externo para permitir que la gerencia comercial ajuste las políticas fácilmente sin requerir intervención del equipo TI.
3. **Sanitización de Datos:** Las pruebas de estrés demostraron que el sistema carece de un filtro preventivo. Es imperativo implementar Expresiones Regulares (Regex) en la capa de entrada (formulario web) para asegurar que los campos mantengan el formato correcto, previniendo caídas del servicio por errores humanos al ingresar datos.

Parte C — Errores y casos borde

- Primera entrada inválida

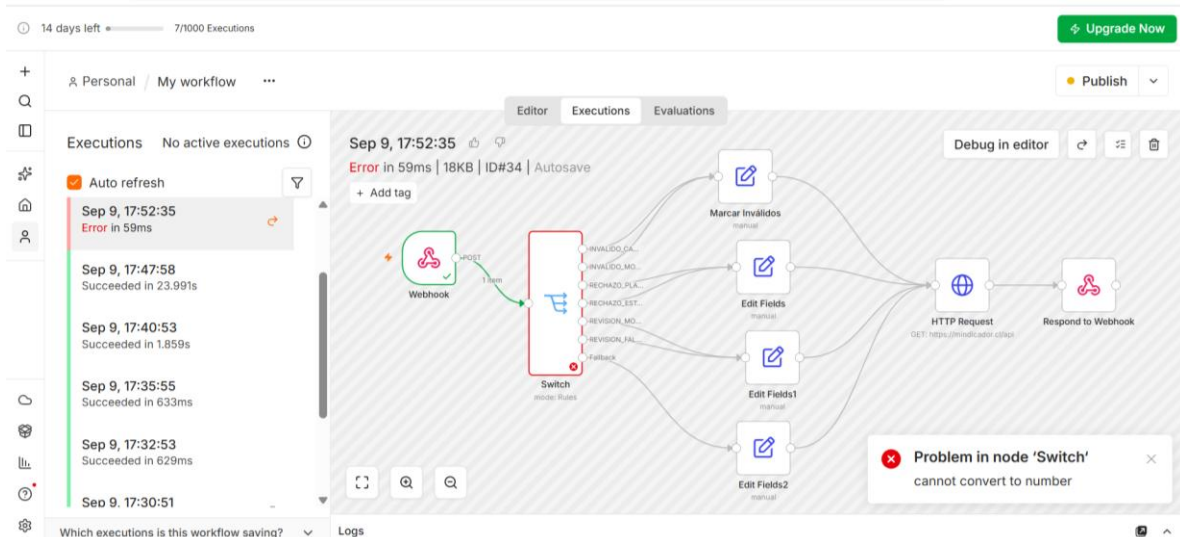


- Segunda entrada inválida



Al ingresar valores fuera de los parámetros lógicos del negocio (como un monto negativo o una cantidad de días que excede las políticas), el nodo switch evalúa las condiciones matemáticas como falsas. Consecuentemente, el sistema deriva la solicitud de forma correcta hacia las rutas de rechazo o de datos inválidos, demostrando que el filtro principal funciona.

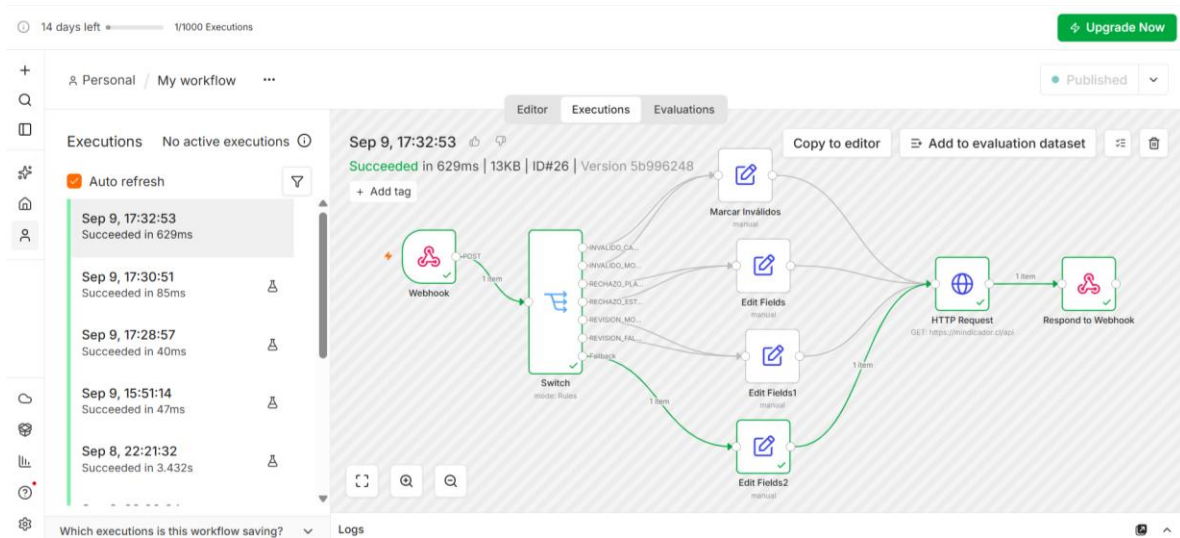
- Tercera entrada



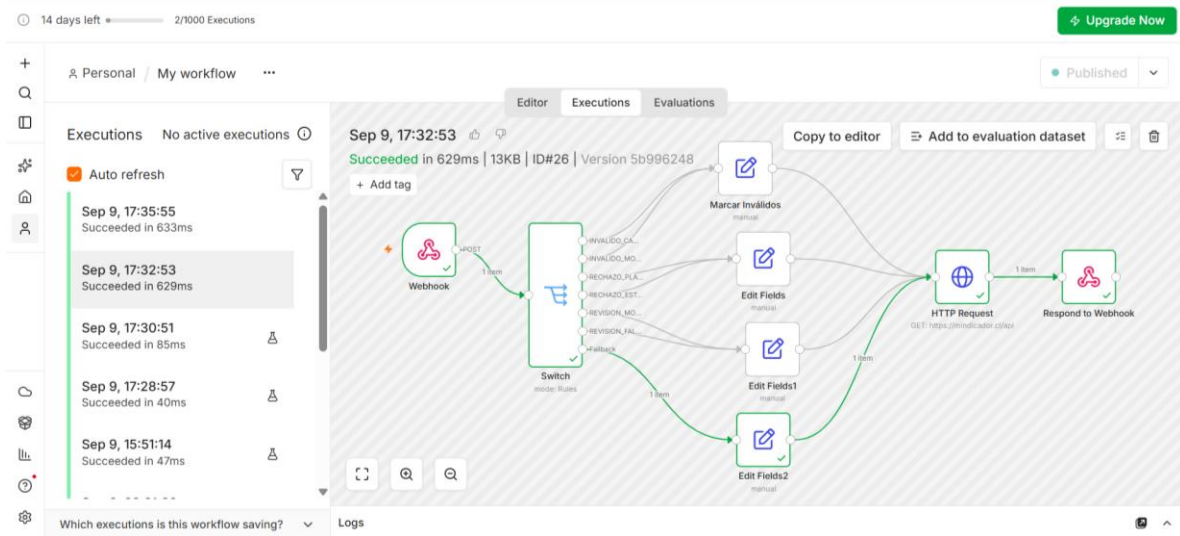
Al enviar el monto escrito con palabras ('veinte mil'), el flujo experimenta un fallo crítico. Esto ocurre porque la función `.toNumber()` intenta aplicar una operación matemática sobre caracteres alfabéticos, arrojando un error (NaN) que detiene la ejecución del nodo.

Solución propuesta: Implementar un nodo previo que utilice expresiones regulares para validar que el campo del monto contenga exclusivamente dígitos antes de pasarlo al Switch.

- Validación del límite U

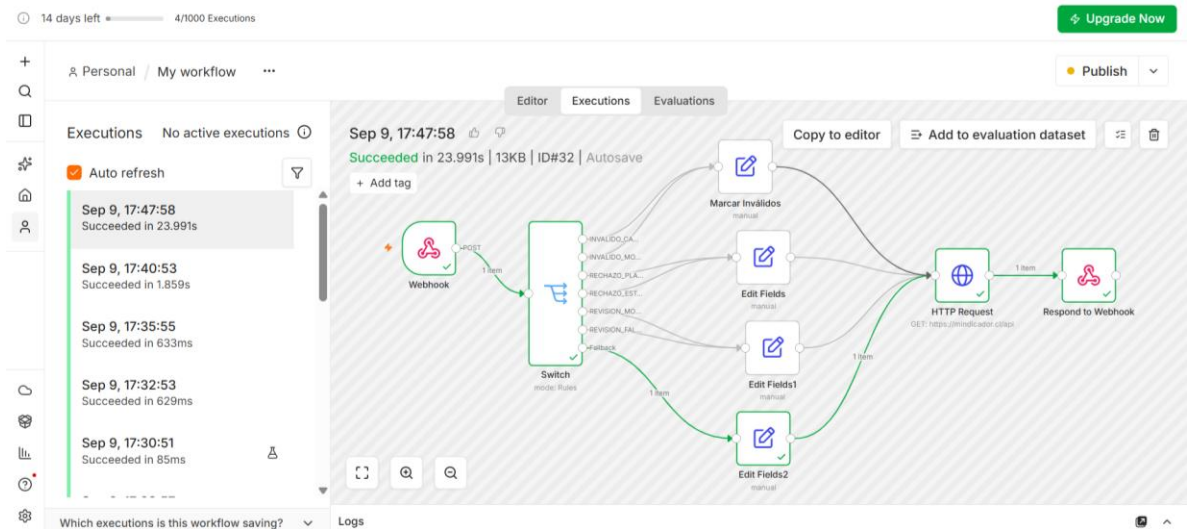


- Validación del límite D



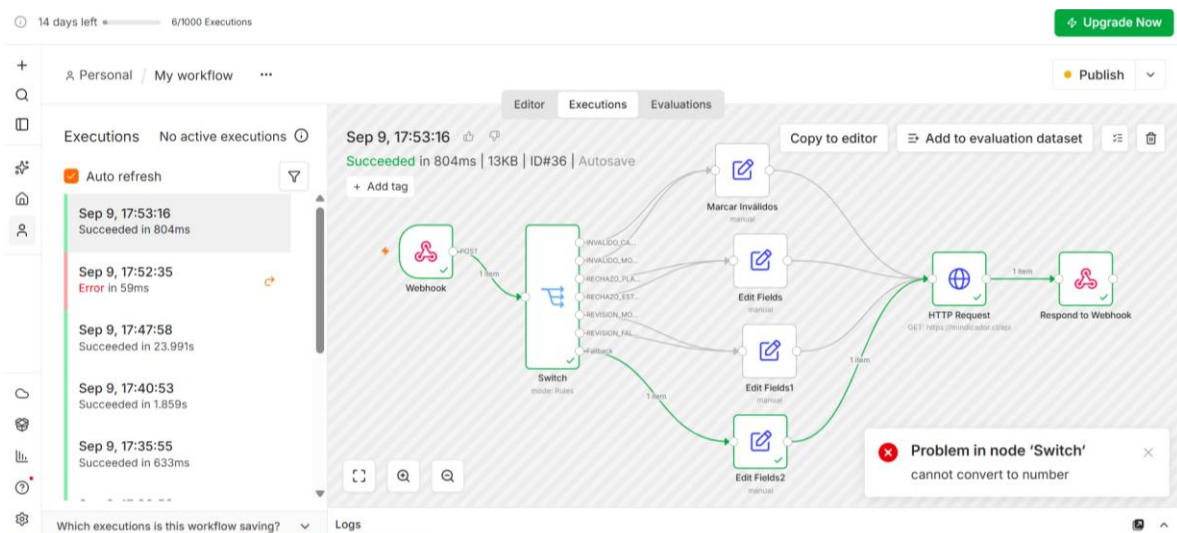
Al inyectar los valores límite exactos (monto de \$50.000 y plazo de 7 días), el flujo procesa las solicitudes por la ruta de aprobación. Esto demuestra matemáticamente que las reglas del nodo Switch están configuradas de manera inclusiva en los bordes, reconociendo correctamente el límite sin generar falsos rechazos.

- El error de tipo string



Al configurar la evaluación del monto como texto (String) en lugar de número (Number), el sistema realiza una comparación alfabética o lexicográfica. Debido a esto, el texto '9990' (que comienza con '9') es erróneamente considerado como 'mayor' que el texto '50000' (que comienza con '5'). Esto provoca que el flujo derive la solicitud por una ruta incorrecta, demostrando la importancia de forzar tipos de datos numéricos (como `.toNumber()`) en integraciones.

- El falso positivo



Al enviar un SKU inexistente ('ZZ-999') con datos numéricos correctos, el flujo lo clasifica como válido y lo procesa exitosamente. Esto es un falso positivo porque el

sistema actual solo realiza validaciones sintácticas (formato de los datos) y no semánticas (veracidad de los datos).

Solución propuesta: Integrar un nodo *HTTP Request* adicional al inicio del flujo que consulte directamente la base de datos o API del inventario real para validar la existencia del SKU antes de evaluarlo.

Bonus

The screenshot displays a workflow editor interface. On the left, a sidebar contains a list of executions for 'My workflow 2'. The main workspace shows a workflow diagram with two nodes: 'Webhook1' (a red circle with a white lightning bolt) and 'Code in Python' (a yellow box with a white lightning bolt). A green arrow labeled '1 item' connects the output of 'Webhook1' to the input of 'Code in Python'. The 'Webhook1' node is highlighted with a green border and a green checkmark, indicating a successful execution. Above the nodes, the status 'Succeeded in 88ms | SKB | ID#56 | Autosave' is displayed. The top right corner features a 'Publish' button. The bottom left corner shows a 'Logs' section with the text 'Which executions is this workflow saving?'.

Execution	Status	Duration
Sep 13, 23:00:58	Succeeded	88ms
Sep 13, 22:57:23	Error	832ms
Sep 13, 22:57:00	Succeeded	34ms
Sep 13, 22:56:12	Succeeded	101ms
Sep 13, 15:50:42	Succeeded	1.909s



Auditoría y Transparencia

- **Nodo Switch:** Es significativamente más fácil de auditar para perfiles de negocio (analistas, gerentes de operaciones). Al ser una interfaz gráfica ("Low-Code"), la lógica de enrutamiento es autodescriptiva y visual.
- **Nodo Code:** Funciona como una "caja negra" para usuarios sin conocimientos de programación. Su auditoría requiere un perfil técnico capaz de interpretar python.

Mantenibilidad y Escalabilidad

- **Nodo Switch:** Su mantenimiento se vuelve complejo y propenso a errores si las reglas de negocio crecen excesivamente o si se requieren operaciones matemáticas complejas entre múltiples variables.
- **Nodo Code:** Altamente mantenible para desarrolladores. Permite refactorización rápida, manejo de excepciones complejas y la inclusión de comentarios técnicos directamente en la lógica.

Conclusión ante cambios mensuales, si la política cambia cada mes, la elección depende de la gobernanza del dato. Si los cambios los define e implementa el equipo de operaciones o negocio, el switch es superior por su agilidad visual. Sin embargo, si el proceso es gestionado por el equipo de TI, el nodo code es la opción definitiva, ya que permite versionar reglas complejas sin alterar visualmente el lienzo del proceso.

